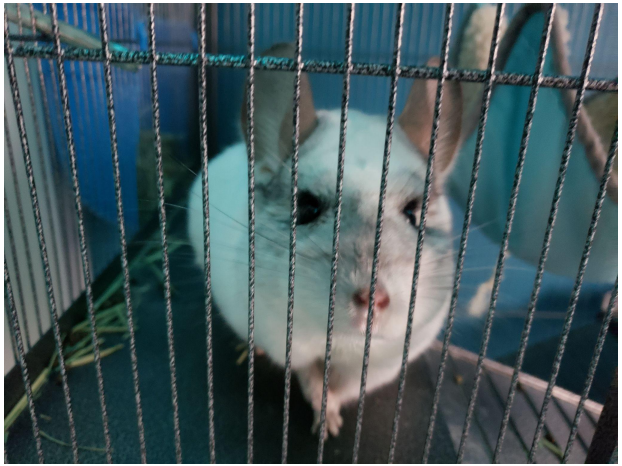
CSCI 4061 - Lab7

10/25/2026

This is Coconut. Coconut is a chinchilla. If you ever get a chance to pet a chinchilla DO IT. They have the softest fur in the history of ever.



Coverage

- Creating Threads
- Mutexes
- Thread Sanitizer

pthreads: Create and Join

```c
#include <pthread.h>
void *worker(void *arg) {
    int id = *(int *) arg;
    printf("hello from thread %d\n", id);
    return NULL;
}
int main(void) {
    pthread_t tids[4];
    int ids[4];
    for (int i = 0; i < 4; i++) {
        ids[i] = i;
        pthread_create(&tids[i], NULL, worker, &ids[i]);
    }
    for (int i = 0; i < 4; i++)
        pthread_join(tids[i], NULL);
    return 0;
}
```

pthread_create

Starts worker(arg) in a new thread. Give each thread its own argument.

pthread_join

Waits for a thread to finish, like waitpid for threads. The second argument can collect its return value.

void *(*fn)(void *)

The thread function takes one void * and returns a void *. It starts running as soon as pthread_create succeeds.

Error checking

pthread_* return 0 or an error number and don't set errno. Use strerror(ret), not perror.

pthreads: Lock and Unlock

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long counter = 0;

void *worker(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&lock);
        counter++;  // critical section
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

counter++ is load, add, store. Without the lock, threads interleave and updates get lost.

Lock right before the shared data, unlock right after. Keep the critical section small.

Every path that locks must unlock. An early return while holding it deadlocks the next thread.

lock and unlock can fail too. Check the return value like any other pthread_* call.

Sanitizers

The compiler adds runtime checks. Recompile, run normally, and get a report at the first bug.

`-fsanitize=address`	Out-of-bounds access, use-after-free, double free, memory leaks
`-fsanitize=undefined`	Undefined behavior: signed overflow, null pointer use, bad shifts
`-fsanitize=thread`	Data races between threads

```
# what this lab's Makefile runs
gcc -Wall -Werror -g -fsanitize=thread -fno-omit-frame-pointer -o sum_files sum_files.c
```

Reading a TSAN Report

counter.c (no lock)

```
1  long counter = 0;
2
3  void *worker(void *arg) {
4      for (int i = 0; i < 1000000; i++)
5          counter++;
6      return NULL;
7  }
```

$ ./counter

```
WARNING: ThreadSanitizer: data race
  Read of size 8 at 0x... by thread T2:
    #0 worker counter.c:5
  Previous write of size 8 at 0x... by thread T1:
    #0 worker counter.c:5
  Location is global 'counter' of size 8
SUMMARY: ThreadSanitizer: data race counter.c:5
```

What

Two threads touch the same memory, at least one writes, and nothing orders them.

Where

One stack trace per thread. Here both point at line 5.

Fix

Lock around the access. Run again: no report.